

TECHNICAL REFERENCE GUIDE

Docker

Under the Hood

A comprehensive, topic-wise walkthrough of Docker's internal architecture — from the CLI and daemon to container networking, kernel isolation, and runtime components.

Topics Covered

14 Sections

Audience

Engineers & DevOps

Version

Docker Engine (current)

Date

August 2026

Table of Contents

01	The Big Picture	3
02	Docker CLI and Docker API	3
03	dockerd, containerd, and runc	4
04	Main Example Used in This Guide	4
05	Container Startup Sequence	5
06	Docker Daemon Startup and Initial Network Setup	7
07	Container Networking Setup	8
08	Networking Traffic Flows	9
09	Sequence Diagram for docker run	10
10	What Happens When You Run docker ps	11
11	Verify the Example Step by Step	11
12	Common Docker Command Flows	13
13	Does Every Tool Require dockerd?	14
14	Final Mental Model	14

The exact implementation varies by Docker Engine version and operating system. On Docker Desktop, the Linux Docker Engine usually runs inside a Linux VM.

1 The Big Picture

Docker is not one single process. A typical command moves through several layers:

```
graph TD; You[You] --> DockerRun[docker run nginx]; DockerRun --> DockerCLI[Docker CLI]; DockerCLI --> APIRequest[Docker Engine API request  
usually via /var/run/docker.sock]; APIRequest --> dockerd[dockerd]; dockerd --> DockerManagement[Docker-specific management]; DockerManagement --> containerd[containerd]; containerd --> ContainerOps[Container lifecycle and image operations]; ContainerOps --> runc[runc]; runc --> LowLevelRuntime[Low-level OCI runtime]; LowLevelRuntime --> LinuxKernel[Linux kernel]; LinuxKernel --> namespaces[namespaces]; LinuxKernel --> cgroups[cgroups]; LinuxKernel --> Capabilities[capabilities and security controls]; LinuxKernel --> networking[networking]; LinuxKernel --> OverlayFS[OverlayFS mounts];
```

You

|

| docker run nginx

▼

Docker CLI

|

| Docker Engine API request

| usually via /var/run/docker.sock

▼

dockerd

|

| Docker-specific management

▼

containerd

|

| Container lifecycle and image operations

▼

runc

|

| Low-level OCI runtime

▼

Linux kernel

|

|— namespaces

|— cgroups

|— capabilities and security controls

|— networking

|— OverlayFS mounts

Responsibilities at a Glance

Component	Simple Meaning	Main Responsibilities
Docker CLI	Terminal client	Sends commands such as <code>docker run</code> , <code>docker ps</code> , and <code>docker stop</code>
dockerd	Docker Engine daemon	Manages images, containers, networks, volumes, ports, and the Docker API
containerd	Container lifecycle manager	Manages image layers, container tasks, processes, and runtime integration
runc	Low-level OCI runtime	Creates and starts the isolated Linux process
Linux kernel	Operating system core	Provides namespaces, cgroups, networking, mounts, and capabilities

2 Docker CLI and Docker API

The `docker` command is the client used from the terminal:

```
docker run nginx
docker ps
docker images
docker build -t my-app .
docker stop my-container
```

The CLI does not create containers by itself. It sends requests to the Docker Engine API. On a typical Linux installation, the request uses:

```
/var/run/docker.sock
```

The basic path is:

```
Docker CLI → Docker Engine API → dockerd
```

For a remote Docker host, the CLI can use a configured remote Docker endpoint instead of the local Unix socket.

3 dockerd, containerd, and runc

3.1 dockerd

`dockerd` is the long-running Docker Engine daemon. It handles Docker-specific features such as:

- Container and image management
- Network creation and configuration
- Volume management
- Published ports
- Docker configuration
- Docker API access
- Communication with `containerd`

Check the daemon on Linux with:

```
systemctl status docker
ps aux | grep dockerd
```

3.2 containerd

`containerd` is a general-purpose container runtime manager. It commonly handles:

- Pulling and storing image layers
- Container metadata
- Container tasks and processes
- Starting and stopping containers
- Communication with OCI runtimes such as `runc`

`containerd` is not Docker-specific. Kubernetes can use `containerd` directly through the Container Runtime Interface.

3.3 runc

`runc` is the low-level OCI runtime. It asks the Linux kernel to create the isolated process and configure:

- Namespaces
- Cgroups
- Capabilities
- Mount points
- The root filesystem
- The initial container process

Key insight: A container is not a lightweight virtual machine. It is an ordinary Linux process isolated and restricted by kernel features.

4 Main Example Used in This Guide

We will use this command throughout the guide:

```
docker run -d --name web -p 8080:80 nginx
```

Option	Meaning
<code>-d</code>	Run in detached/background mode
<code>--name web</code>	Name the container <i>web</i>
<code>-p 8080:80</code>	Map host port 8080 to container port 80
<code>nginx</code>	Use the nginx image (normally nginx:latest)

The complete high-level flow is:

Shell → Docker CLI → dockerd

- Check image, create metadata, configure network and ports
- containerd → Prepare filesystem and container task
- runc → Create isolated process
- Linux kernel → Start nginx
- Running container

5 Container Startup Sequence

Step 1 – The shell starts the Docker CLI

The shell passes the arguments (`run` , `-d` , `--name web` , `-p 8080:80` , `nginx`) to the CLI, which converts them into Docker Engine API requests.

Step 2 – The CLI contacts dockerd

```
Docker CLI → Docker API → /var/run/docker.sock → dockerd
```

Step 3 – dockerd checks the image

```
If the image exists: use the local image
If the image is missing:
    dockerd → configured registry → download image layers
```

A container image contains read-only filesystem layers (base OS, packages, application, configuration). The real number and content of layers depend on the image and platform.

Step 4 – dockerd asks containerd to create the container

The request can contain: image name, container name, environment variables, volumes and bind mounts, network settings, port mappings, resource limits, and entrypoint/command.

Step 5 – The container filesystem is prepared

On Linux, OverlayFS commonly combines image layers with a writable container layer:

```
Read-only image layers
      +
Writable container layer
      =
Container root filesystem
```

The image remains unchanged when the container writes files. Removing a container normally removes its writable layer but not the image. Named volumes have separate retention behavior.

Step 6 – Networking is prepared

Docker creates or connects the container to a network, assigns an IP address, configures routes and DNS, and applies port and NAT rules. See Section 7 for the detailed networking flow.

Step 7 – containerd invokes runc

```
containerd → runc → Linux kernel
```

Step 8 – The Linux kernel provides isolation

Namespace	What it isolates
PID	Own process tree
Network	Own interfaces, IP addresses, and routes
Mount	Own filesystem mounts
UTS	Own hostname
IPC	Isolated inter-process communication

Cgroups account for and limit resources such as CPU, memory, block I/O, and number of processes. Example:

```
docker run --memory=512m nginx
```

Security controls include Linux capabilities, Seccomp, AppArmor, SELinux (when enabled), read-only filesystems, and user namespaces (when configured).

Step 9 – The container process starts

Nginx starts as a normal Linux process inside the configured namespaces and cgroups. A simplified process relationship:

```
dockerd
├─ containerd
│   └─ containerd-shim
│       └─ nginx
```

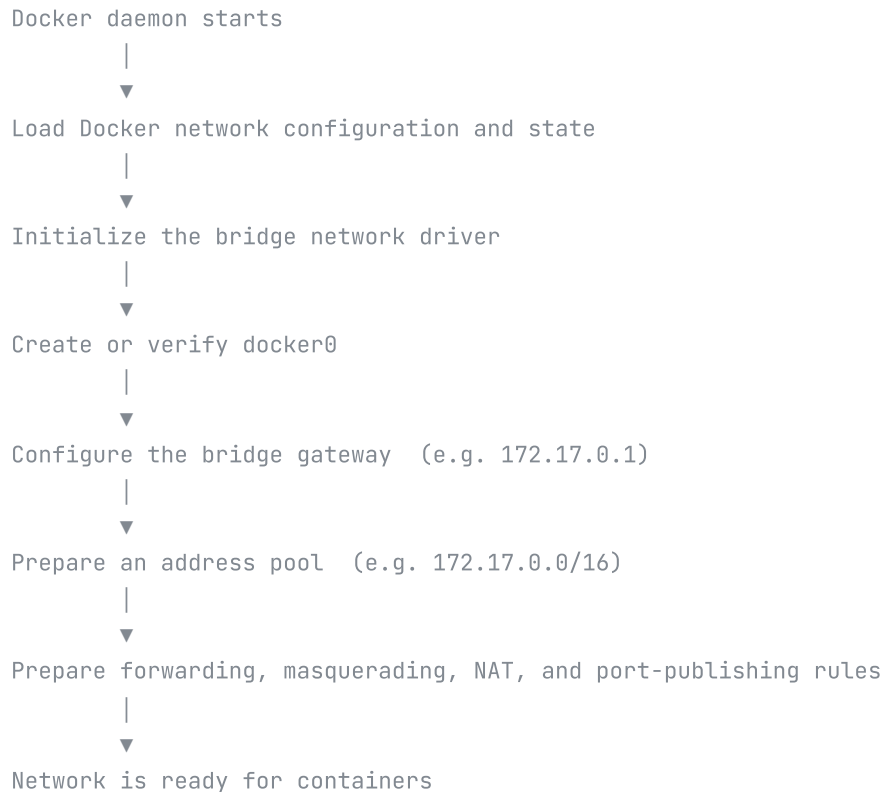
Step 10 – The CLI returns the result

Because detached mode was used, Docker returns a container ID. Useful follow-up commands:

```
docker ps
docker inspect web
docker logs web
docker exec -it web sh
```

6 Docker Daemon Startup and Initial Network Setup

Before a container starts, Docker initializes its network controller when the daemon starts or when networks are first needed.



The actual subnet, firewall backend, and implementation can differ. Linux Docker may use **iptables** or **nftables**. Docker Desktop uses networking through its Linux VM.

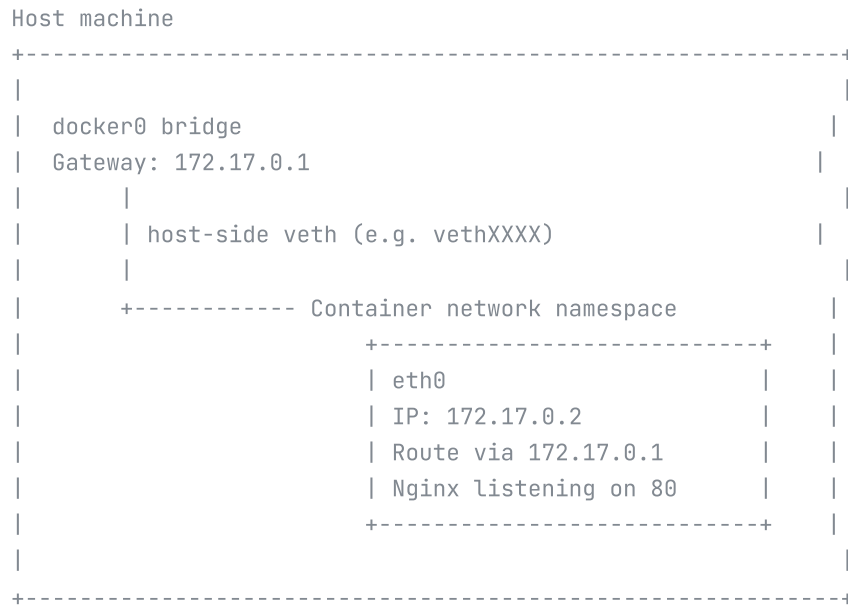
7 Container Networking Setup

For `docker run -d --name web -p 8080:80 nginx`, Docker typically performs these networking steps:

1. Create a network namespace
2. Create a virtual Ethernet pair
3. Attach the host side to `docker0` or another Docker network
4. Move the container side into the container namespace
5. Rename the container-side interface to `eth0`
6. Assign a container IP address (e.g. `172.17.0.2`)
7. Add a default route through `172.17.0.1`
8. Configure DNS

- 9. Add outbound NAT / masquerading rules
- 10. Add the published port rule for `8080:80`

Network Component Diagram



Main Networking Components

Component	Purpose
Network namespace	Gives the container its own interfaces, routes, and ports
veth pair	Virtual cable between the host and container namespaces
<code>docker0</code> bridge	Virtual switch connecting containers on the default bridge
Container IP	Address allocated from the Docker network subnet
Gateway	Bridge address used for traffic leaving the container network
NAT/masquerading	Rewrites container source addresses for outbound traffic
Port publishing	Forwards a host port to a container port
DNS configuration	Allows name resolution inside the container

8 Networking Traffic Flows

8.1 Container to Container



On user-defined Docker networks, Docker commonly provides embedded DNS at [127.0.0.11](#) , allowing containers to reach one another by service or container name.

8.2 Container to Internet

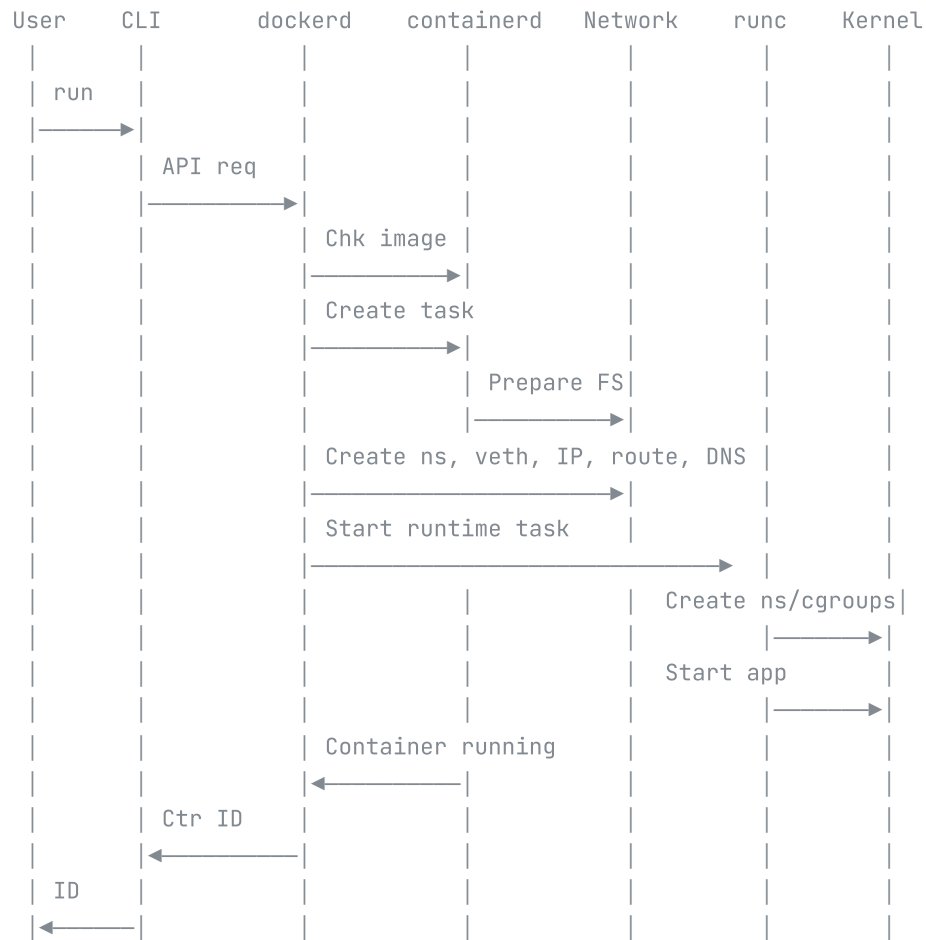
```
Container process (Source: 172.17.0.2:40000, Dest: external-site:443)
  → Container eth0
  → Docker bridge
  → Host firewall and NAT (source rewritten to host IP)
  → Internet
  ← Return traffic matched by connection tracking, translated back
```

8.3 Published Port

```
Browser → localhost:8080
  → Host port 8080
  → Docker forwarding / DNAT
  → Container IP 172.17.0.2:80
  → Nginx process

Response (reverse path):
  Nginx:80 → Container eth0 → Docker bridge
  → Reverse NAT → Host port 8080 → Browser
```

Docker can implement published ports using firewall NAT rules and, in some configurations, a userland proxy.



10 What Happens When You Run `docker ps`

The flow is much shorter than `docker run` :

```
User → docker ps → Docker CLI
  → GET /containers/json → dockerd
  → Read Docker metadata and current task state
  → Return running containers
  → Filter and format result → Terminal table
```

Example output:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	
NAMES					
a1b2c3d4e5f6	nginx	"/docker-entrypoint..."	Up 10 seconds	0.0.0.0:8080→80/tcp	web

What `docker ps` Does NOT Normally Do

- Pull an image
- Create a new container
- Invoke `runc` to create a new container
- Create a network namespace
- Allocate a new IP address
- Create a new veth pair
- Recreate the published port rule

The `PORTS` column displays existing port-publishing metadata. It does not create the port mapping at that moment. Use `docker ps -a` to include stopped containers.

11 Verify the Example Step by Step

11.1 Start the container

```
docker run -d --name web -p 8080:80 nginx
```

11.2 Check the Docker-level state

```
docker ps
```

Expected output includes: `web nginx Up ... 0.0.0.0:8080→80/tcp`

11.3 Inspect container configuration

```
docker inspect web
```

Useful fields: `State.Status` , `NetworkSettings.Networks.bridge.IPAddress` , `NetworkSettings.Ports` , `Mounts` , `Config.Image` , `Config.Cmd`

```
# Get only the container IP:
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web
# Result: 172.17.0.2 (may differ on your system)
```

11.4 Test the published port

```
curl http://localhost:8080
```

Path: `curl` → host port 8080 → Docker port forwarding → container IP:80 → nginx

11.5 Check logs

```
docker logs web
```

11.6 Enter the container

```
docker exec -it web sh

# Inside the container:
hostname
cat /etc/hosts
cat /etc/resolv.conf
ip addr
ip route
```

11.7 Inspect Docker networking

```
docker network ls
docker network inspect bridge
ip addr show docker0
bridge link
```

11.8 Clean up

```
docker stop web
```

```
docker rm web
```

```
# The nginx image normally remains:
```

```
docker images nginx
```

```
# Remove the image only if needed:
```

```
docker rmi nginx
```

docker ps

docker CLI → dockerd → container state → running containers → CLI

Lists containers and does not start a new one.

docker stop web

docker CLI → dockerd → containerd → container process

Docker normally sends `SIGTERM` first. If the process does not stop within the timeout, Docker can send `SIGKILL`.

docker rm web

docker CLI → dockerd → remove container metadata and writable layer

The image normally remains. Named volumes normally remain unless explicitly removed.

docker pull nginx

docker CLI → dockerd → registry → image manifest and layers → local image store

Modern Docker installations commonly involve `containerd` in image transfer and storage.

docker build

A modern build commonly uses BuildKit:

docker CLI → dockerd → BuildKit → Dockerfile steps → image layers

```
FROM nginx
COPY index.html /usr/share/nginx/html/
```

The build creates new layers. The original `nginx` image is not modified.

docker exec

```
docker exec -it web sh
```

This does not create a new container. It starts another process inside the existing container's namespaces and cgroups:

```
docker CLI → dockerd → containerd → runtime exec → sh in existing container
```

13 Does Every Tool Require dockerd?

With standard Docker Engine:

```
docker CLI → dockerd → containerd → runc → Linux kernel
```

The Docker CLI normally requires [dockerd](#) . Other tools can use lower layers directly:

```
Kubernetes → containerd → runc
nerdctl     → containerd → runc
Podman      → OCI runtime → Linux kernel
```

Component	Role
Docker CLI	Command-line client
dockerd	Docker Engine and Docker API
containerd	Container and image lifecycle manager
runc	Low-level OCI runtime
Linux kernel	Isolation, resource controls, networking, mounts, and process execution

For docker run

CLI

- dockerd
- image and container management
- containerd
- filesystem and task preparation
- network setup
- runc
- Linux kernel isolation
- application process starts

For docker ps

CLI

- dockerd
- read existing container state
- filter running containers
- display the table

The most important distinction:

`docker run` creates and starts the container.

`docker ps` only reports the container that already exists.